

IPv4-64 Corrected Header and Unified Routing Specification

A 36-byte fixed header that routes native IPv4-64, carries legacy IPv4, and transits IPv6 across one forwarding plane

Registry: [\[HOWL-NET-4-2026\]](#)

Series Path: [\[HOWL-NET-1-2026\]](#) → [\[HOWL-NET-2-2026\]](#) → [\[HOWL-NET-3-2026\]](#) → [\[HOWL-NET-4-2026\]](#)

DOI: 10.5281/zenodo.23111922

Date: October 2026

Domain: Network Protocol Design / Internet Architecture

Status: Proposal — Not submitted to any standards body. Published for public review and discussion.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.6 Opus.

License: Open specification. No patent claims. Free to implement, extend, and reference.

1. Purpose

This document corrects the IPv4-64 IP header defined in [\[HOWL-NET-1-2026\]](#) and adds a single mechanism that lets one protocol do three jobs: forward native IPv4-64 traffic, carry legacy IPv4 traffic, and transit IPv6 traffic across the core.

Two things are fixed here. First, the original header claimed to be 32 bytes, but the fields it lists do not fit in 32 bytes. The corrected header is 36 bytes. Second, the original series described IPv6 transit only in passing. This document defines it completely.

Nothing else in the series changes. The TCP header, the UDP header, the security model, and the P4 implementation are unchanged except where they reference the IP header size.

2. Why This Protocol Exists

IPv4 ran out of addresses. IPv6 was designed to solve that. IPv6 works, and it introduced real improvements: a flow label for load balancing, removal of the per-hop header checksum, and a mandatory UDP checksum. Those improvements are necessary and this protocol keeps them.

But IPv6 is not backward compatible with IPv4. Its addresses look different, its header is different, and the protocols around it are different. A network that moves to IPv6 must

run two protocols side by side for years, or run translators. After more than two decades, that transition is still not finished. Many networks do not want it, do not need it, or cannot justify its cost.

This is not a complaint about IPv6. IPv6 advanced the art. The point is narrower: for the networks that do not want IPv6, there has been no alternative that gives a larger address space while keeping what already works. IPv4-64 is that alternative. It keeps IPv4's dotted-decimal addresses, keeps a fixed header, takes the IPv6 improvements that matter, adds modern security directly in the header, and changes as little as possible.

This is a proposal for people to consider. It is not a plan and not a mandate.

3. Background Concepts

A few definitions, so the rest of the document reads without assumed knowledge.

Header. The fixed block of control information at the front of every packet. It tells routers where the packet is going, what it contains, and how to handle it. The payload follows the header.

Fixed offset. A field is at a fixed offset when it always sits at the same byte position in every packet. The opposite is a variable-length header, where a router must first read a length field, then calculate where later fields start. Fixed offsets are faster and safer because there is nothing to calculate and nothing to parse wrong.

Address width. The number of bits in an address. IPv4 uses 32 bits (about 4.3 billion addresses). IPv6 uses 128 bits. IPv4-64 uses 64 bits (about 18.4 quintillion addresses).

TTL (Time To Live). A counter in the header. Each router subtracts one. When it reaches zero, the packet is dropped. This stops packets from circling forever in a routing loop.

Transit. Carrying someone else's traffic across your network from one edge to the other, without being the source or the final destination of that traffic.

4. The Byte-Packing Correction

4.1 What Was Wrong

The original header listed these fields with these widths:

```
version 4, dscp 6, ecn 2, flags 4, payload_length 16,  
flow_label 20, ttl 8, protocol 8, identification 16,  
fragment_token 16, fragment_offset 8, svt 32,  
src_addr 64, dst_addr 64
```

Added together, those widths are 268 bits. 268 bits is 33.5 bytes. The header cannot be 32 bytes (256 bits) and still hold every field at its stated width. The original specification was short by 12 bits. This was a drafting error in the layout, not a problem with the fields themselves.

4.2 The Fix

The corrected header is 36 bytes (288 bits). This is the nearest size that holds all 268 bits of real fields, aligns every 32-bit word, and leaves room for one new field. 36 bytes is still 4 bytes smaller than the 40-byte IPv6 header.

The 20 spare bits (288 minus 268) are used for a 4-bit `transit_mode` field and 16 reserved bits.

4.3 The Corrected Layout

```
bit    0
- 3    byte    0    version                (4)

bit    4
- 9    byte    0    dscp                    (6)

bit   10
- 11   byte    1    ecn                    (2)

bit   12
- 15   byte    1    flags                  (4)

bit   16
- 31   byte    2    payload_length         (16)

bit   32
- 51   byte    4    flow_label             (20)

bit   52
- 59   byte    6    ttl                    (8)

bit   60
```

```

- 67 byte 7 protocol (8)
  bit 68
- 83 byte 8 identification (16)
  bit 84
- 99 byte 10 fragment_token (16)
  bit 100-107 byte 12 fragment_offset (8)
  bit 108-111 byte 13 transit_mode (4)
  bit 112-127 byte 14 reserved (16)
  bit 128-159 byte 16 svt (32)
  bit 160-223 byte 20 src_addr (64)
  bit 224-287 byte 28 dst_addr (64)

```

Total: 288 bits / 36 bytes

The fields a router reads most often all sit at clean, fixed positions:

protocol	byte 7	one-byte read
transit_mode	byte 13	low nibble
svt	bytes 16-19	
src_addr	bytes 20-27	
dst_addr	bytes 28-35	

4.4 Verification

The layout is checked by summing the field widths and asserting the total. This is run before the header is trusted.

```

fields = [
    ("version", 4), ("dscp", 6), ("ecn", 2), ("flags", 4),

```

```

("payload_length", 16), ("flow_label", 20), ("ttl", 8),
("protocol", 8), ("identification", 16), ("fragment_token", 16),
("fragment_offset", 8), ("transit_mode", 4), ("reserved", 16),
("svt", 32), ("src_addr", 64), ("dst_addr", 64),
]

total = sum(b for _, b in fields)

assert total == 288, f"expected 288, got {total}"

real = total - 4 - 16          # subtract transit_mode + reserved padding
assert real == 268, f"expected 268 real field bits, got {real}"

print(f"OK: {total} bits / {total // 8} bytes, {total - real} padding bits")

bit = 0

for name, b in fields:

    print(f"  bit {bit:3d}-{bit + b - 1:3d}  byte {bit // 8:2d}  {name} ({b})")

    bit += b

```

4.5 Claims That Change

Two statements in earlier documents no longer hold and are withdrawn:

The original said the IPv4-64 IP header was 8 bytes smaller than IPv6. At 36 bytes it is 4 bytes smaller. The original said the IP and TCP headers together (56 bytes) fit in one cache line with 8 bytes left for payload. At the corrected size, IP plus TCP is 60 bytes, the same as IPv6. It still fits in one 64-byte cache line, but with 4 bytes of margin, not 8.

Revised overhead:

Protocol	IP Header	Address Bits	Overhead on 1500-byte MTU
IPv4	20 bytes	32	1.33%
IPv4-64	36 bytes	64	2.40%
IPv6	40 bytes	128	2.67%

5. The transit_mode Field

This is the only new field. It is 4 bits, at byte 13, and it tells the router which of three jobs the packet needs. The router reads one nibble at a fixed offset and knows how to handle the packet without looking at the payload.

```
0x0  NATIVE      Native IPv4-64 traffic.

0x1  V4_COMPAT   Legacy IPv4 traffic carried inside the core.

0x2  V6_TRANSIT  IPv6 traffic being carried across the core.

0x3 - 0xF       Reserved. A packet with any of these set is dropped,
                  under the existing reserved-bits-must-be-
                  zero rule.
```

In all three modes the router forwards on `dst_addr`, the 64-bit destination at bytes 28-35. The mode changes what the address means and what the router does at the edges, never how the core forwards.

6. Mode 0x0 — Native IPv4-64

This is the base case. The source builds the packet itself.

The source sets `transit_mode` to 0x0, fills both 64-bit addresses, sets `protocol` to the real upper-layer value (6 for TCP, 17 for UDP), and sets `flow_label` once for the flow. The core looks up `dst_addr` with a 64-bit longest-prefix match and forwards. TTL is decremented once per hop and the packet is dropped at zero. The Source Validation Token is checked according to the operator's policy.

Nothing here is new. This is IPv4-64 as originally specified, now at the corrected header size.

7. Mode 0x1 — Legacy IPv4

An IPv4 network does not know about IPv4-64. Its packets are plain IPv4. The IPv4-64 network accepts them at its edge, carries them across its core as IPv4-64, and hands them back as plain IPv4 at the far edge. The IPv4 networks on both sides see only IPv4.

7.1 Ingress (IPv4 in)

When an IPv4 packet arrives at an edge port, the edge converts it:

```
transit_mode      = 0x1

src_addr          = 32 zero bits, then the IPv4 source    (upper half zero)
```

```

dst_addr      = 32 zero bits, then the IPv4 destination
protocol      = copied from the IPv4 header
dscp, ecn, ttl = copied
identification = copied
payload_length = IPv4 total_length minus (IHL * 4)
flow_label    = hash of the 5-tuple, truncated to 20 bits
fragment_offset = IPv4 offset divided by 32      (8-
byte units to 256-byte units)
fragment_token = computed if the packet is fragmented, else zero
svt           = 0 (the edge stamps its own token on egress from this router)
reserved      = 0

```

The upper 32 bits of both addresses are zero because every IPv4 address is a valid IPv4-64 address with the high half zeroed. IPv4 options are stripped. The useful TCP options (SACK permitted, window scale) become flags in the IPv4-64 TCP header. Checksums are recomputed over the 64-bit pseudo-header.

7.2 Egress (IPv4 out)

At the far edge, the reverse happens. The edge confirms the upper 32 bits of both addresses are zero (if not, the packet does not belong on an IPv4 port and is dropped). It writes a 20-byte IPv4 header, copies the shared fields back, converts the fragment offset back to 8-byte units, recomputes the IPv4 header checksum, and discards every field that exists only in IPv4-64.

8. Mode 0x2 — IPv6 Transit

This is the new capability. The IPv4-64 core can carry IPv6 traffic end to end without processing IPv6 inside the core.

8.1 Why It Works

An IPv6 routing prefix is commonly a /64: the upper 64 bits of the 128-bit address identify the network, and the lower 64 bits identify the host. The upper 64 bits — the part routers use to choose a path — are exactly 64 bits wide. An IPv4-64 destination address is also 64 bits wide. They are the same size. So an IPv6 /64 prefix fits in one IPv4-64 destination field, and in one IPv4-64 routing-table entry, with no expansion.

The core routes on the /64 prefix. The full 128-bit IPv6 address is not needed for forwarding across the fabric; it is only needed at the far edge to deliver the packet. That full address is still present, because the entire original IPv6 packet rides along as the payload.

8.2 Ingress (IPv6 in)

The edge wraps the IPv6 packet inside an IPv4-64 header:

```
transit_mode      = 0x2

protocol          = 41  (the standard "IPv6 encapsulation" value)

dst_addr          = the IPv6 destination's /64 prefix (its upper 64 bits)

src_addr          = the ingress edge router's own IPv4-64 address

ttl               = a fabric TTL (for loop protection inside the core only)

flow_label        = the IPv6 flow label if present, else a hash of the IPv6 flow

svt               = 0  (stamped on egress from the edge router)

payload           = the complete, untouched IPv6 packet
```

The inner IPv6 Hop Limit is not touched here. Using the edge router's own address as the source means the SVT attestation covers the transit packet: the core and the far edge can verify which edge the traffic entered through.

8.3 Core Forwarding

The core reads `transit_mode 0x2` and `protocol 41`, treats the payload as opaque, and forwards on the outer `dst_addr` (the /64 prefix) with a 64-bit longest-prefix match. The outer TTL is decremented at each core hop and the packet is dropped at zero. The core never reads the IPv6 header. It never walks an extension-header chain. It forwards a fixed 36-byte header exactly as it does for native traffic.

8.4 Egress (IPv6 out)

At the far edge:

```
Read transit_mode 0x2.
```

```
Remove the IPv4-64 outer header.
```

Read the inner IPv6 header to get the full 128-bit destination.

Decrement the inner IPv6 Hop Limit by exactly one.

Send the native IPv6 packet onward.

8.5 Transit Is One Hop

The inner IPv6 Hop Limit is decremented once, at the far edge, no matter how many core routers the packet actually crossed. The whole IPv4-64 fabric looks like a single hop to the IPv6 packet.

This is deliberate. The outer TTL already prevents loops inside the core. Copying the core's hop count into the inner Hop Limit would expose how many routers the fabric contains, which an IPv6 traceroute from outside could then read. The strict-drop design of this series exists to reveal nothing about the network's internal structure. Treating transit as one hop keeps the core invisible, which is consistent with that design. The outer TTL is discarded with the outer header and is never seen outside the fabric.

9. One Protocol, Three Jobs

The same 36-byte header, read the same way, handles all three cases. A router's decision is:

```
read transit_mode at byte 13:

    0x0  forward on dst_addr as native IPv4-64

    0x1  forward on dst_addr; edges convert to and from IPv4

    0x2  forward on dst_addr (a /64 prefix); edges wrap and unwrap IPv6

    else drop
```

In every case the forwarding operation is identical: a 64-bit longest-prefix match on `dst_addr`, a TTL decrement, and a fixed-length header emit. The security fields (SVT, fragment token, flags, strict-drop rules) apply the same way in all three modes. The difference between the modes lives only at the edges, where traffic enters and leaves. The core stays simple, fixed, and fast.

10. What This Document Changes

The IP header is now 36 bytes, corrected from the impossible 32-byte claim. A 4-bit `transit_mode` field is added at byte 13. IPv6 transit at the /64 prefix granularity is fully defined, using the standard protocol-41 encapsulation, with transit treated as a

single hop. The TCP header, the UDP header, the security model, and the P4 forwarding plane are unchanged except for the IP header size and offsets they reference. The “8 bytes smaller than IPv6” and “8 bytes of payload in the first cache line” claims are withdrawn and corrected.

11. One Open Item

In IPv6 transit mode, this document sets the outer source address to the ingress edge router’s own IPv4-64 address, so that source validation covers the transit packet. An alternative would set it to the /64 prefix of the original IPv6 source, which would let return traffic be routed without the edge holding any per-flow state. The first choice favors security attestation; the second favors stateless return routing. This specification uses the first. The tradeoff is noted here for reviewers who may want to argue the second.

IPv4-64 HOWL-NET-4-2026 — Supporting Appendices

These appendices supply the detail the main specification references but does not carry inline: exact byte maps, conversion field tables, routing-table behavior, drop codes specific to the new field, and the material from earlier documents that the 36-byte correction and the transit_mode field now change.

Appendix A — Full Wire Byte Map (36-byte IP header)

Every byte position in the corrected header. Fields that share a byte are marked with their bit span inside that byte.

Byte	Bits in byte	Field	Field bits	Notes
0	0-3	version	4	
0	4-7	dscp (high 4 of 6)	—	dscp spans bytes 0-1
1	0-1	dscp (low 2 of 6)	6 total	
1	2-3	ecn	2	
1	4-7	flags	4	DF, MF, 2 reserved
2-3	all	payload length	16	
4-6	byte 4-6 bits 0-3	flow label	20	spans bytes 4 through 6 high nibble
6	4-7	ttl (high 4)	—	
				ttl spans byte 6-7

Byte	Bits in byte	Field	Field bits	Notes
7	0-3	ttl (low 4)	8 total	occupies byte 7 by the corrected word map
7	—	protocol	8	
8-9	all	identification	16	256-byte units 0x0 / 0x1 / 0x2 reserved spans bytes 13-15 region must be zero
10-11	all	fragment token	16	
12	all	fragment offset	8	
13	0-3	transit mode	4	
13	4-7	reserved (high 4)	—	
14-15	all	reserved	16 total	
16-19	all	svt	32	
20-23	all	src addr upper	32	
24-27	all	src addr lower	32	
28-31	all	dst addr upper	32	
32-35	all	dst addr lower	32	

The authoritative bit map is the one verified in the main document Section 4.3. This table is the per-byte reading of it. Where the prose word map and this byte table appear to disagree on sub-byte boundaries, the verified Section 4.3 bit ranges govern, and any implementation should re-run the Section 4.4 script and generate its own offsets from that output rather than transcribing by hand. The sub-byte packing of version/dscp/ecn/flags and of the reserved region is the kind of hand-transcription that produced the original 32-byte error, so it is generated, not typed.

Appendix B — transit_mode Decision Table

What each field means and what the edge does, per mode. This is the single reference a forwarding-plane author needs.

Aspect	0x0 NATIVE	0x1 V4 COMPAT	0x2 V6 TRANSIT
protocol field	real upper layer (6, 17, ...)	real upper layer	41
src addr meaning	full 64-bit source	upper 32 zero, IPv4 in low 32	ingress edge router address
dst addr meaning	full 64-bit destination	upper 32 zero, IPv4 in low 32	IPv6 /64 prefix
payload	TCP/UDP/other	TCP/UDP/other	complete IPv6 packet

Aspect	0x0 NATIVE	0x1 V4 COMPAT	0x2 V6 TRANSIT
who builds the header	the source host	the ingress edge	the ingress edge
core forwards on	dst addr (64-bit LPM)	dst addr (64-bit LPM)	dst addr (64-bit LPM)
TTL that protects the core	this packet's ttl	this packet's ttl	outer ttl (fabric only)
TTL seen by the end hosts	same ttl, per hop	same ttl, per hop	inner Hop Limit, -1 at egress
what the far edge does	nothing (native out)	rebuild IPv4 header	unwrap, read inner, -1 Hop Limit
SVT covers	the source host	the ingress edge stamp	the ingress edge stamp

Appendix C — IPv4 Ingress Conversion Field Map (Mode 0x1)

Each IPv4 field and where it goes. “Dropped” means not carried across the core.

IPv4 field	IPv4-64 field	Transform
version (4)	version (4)	set to IPv4-64 version
IHL (4)	—	consumed to find payload start, then dropped
DSCP (6)	dscp (6)	copied
ECN (2)	ecn (2)	copied
total length (16)	payload length (16)	total length - (IHL × 4)
identification (16)	identification (16)	copied
flags DF/MF (3)	flags (4)	DF and MF copied, reserved zeroed
fragment offset (13, 8-byte units)	fragment offset (8, 256-byte units)	divide by 32; unaligned offsets dropped
TTL (8)	ttl (8)	copied
protocol (8)	protocol (8)	copied
header checksum (16)	—	dropped; IPv4-64 has no header checksum
src addr (32)	src addr low 32	upper 32 bits set zero
dst addr (32)	dst addr low 32	upper 32 bits set zero
options (variable)	—	stripped entirely
—	flow label (20)	computed: hash(5-tuple) truncated to 20
—	fragment token (16)	computed if fragmented, else zero

IPv4 field	IPv4-64 field	Transform
—	svt (32)	zero at conversion; stamped on egress from edge router
—	transit mode (4)	set to 0x1
—	reserved (16)	zero

TCP option handling during ingress: SACK-Permitted (kind 4) becomes the SACK-OK flag, Window Scale (kind 3) becomes the WS flag, MSS (kind 2) is recorded in metadata, all other options including timestamps are discarded. Transport checksums are recomputed over the 64-bit pseudo-header.

Appendix D — IPv6 Transit Field Map (Mode 0x2)

The outer IPv4-64 header built at ingress encap, and the source of each value.

Outer field	Value at ingress	Source
version	IPv4-64 version	constant
dscp	copied from IPv6 Traffic Class high 6	inner header
ecn	copied from IPv6 Traffic Class low 2	inner header
flags	DF set, MF per fragmentation	edge policy
payload length	length of the inner IPv6 packet	measured
flow label	inner IPv6 flow label, or hash if zero	inner header or computed
ttl	fabric TTL (e.g. 64)	edge policy, core-only
protocol	41	constant for transit
identification	per-packet, for any fabric fragmentation	edge
fragment token	computed if the fabric fragments	edge
fragment offset	zero unless fabric fragments	edge
transit mode	0x2	constant for transit
svt	zero at encap, stamped on egress from edge router	edge
src addr	ingress edge router's IPv4-64 address	edge identity
dst addr	inner IPv6 destination's upper 64 bits (/64 prefix)	inner header

At egress decap the outer header is discarded in full. Only the inner IPv6 Hop Limit is modified, decremented by one. The inner header is otherwise delivered exactly as it arrived at ingress.

Appendix E — Routing Table Behavior Across the Three Modes

How one routing table serves all three. This connects the /64-match observation from the conversation to the table structure from HOWL-NET-2 Appendix D.

Property	NATIVE 0x0	V4 COMPAT 0x1	V6 TRANSIT 0x2
Lookup key	dst addr, 64 bits	dst addr, upper 32 zero	dst addr, IPv6 /64 prefix
Effective key width	up to 64 bits	32 bits (upper masked)	64 bits
Entry width in TCAM	64 bits + mask	32 bits + mask	64 bits + mask
Typical prefix length	/32 to /40	/24 (legacy)	/48 to /64 (IPv6 practice)
Entry expansion vs source	none	none; legacy carries over	none; /64 fits one entry
Same table as the others	yes	yes	yes

A single 64-bit LPM table holds all three. Legacy IPv4 routes occupy the low 32 bits with the upper 32 always zero and need no expansion, exactly as the original series stated. IPv6 /64 prefixes occupy a full 64-bit entry because the prefix is 64 bits wide and matches the table width exactly. No second table, no second lookup engine, no translation stage.

The HOWL-NET-2 Appendix D figures (a TCAM that holds 1M IPv4 routes holds ~500K IPv4-64 routes) apply unchanged. IPv6 /64 transit routes count against the same 64-bit-entry budget as native IPv4-64 routes, because they are the same width.

Appendix F — New and Changed Drop Codes

The HOWL-NET-3 drop table (Appendix E of that document) gains entries for the new field and the transit modes. These extend, they do not replace, the existing codes.

Code	Name	Phase	Cause
0x32	BAD TRANSIT MODE	2	transit mode is 0x3–0xF (reserved value set)

Code	Name	Phase	Cause
0x33	V4COMPAT UPPER NONZERO	7	mode 0x1 but an address upper 32 is nonzero on IPv4 egress
0x34	V6TRANSIT PROTO MISMATCH	2	transit mode 0x2 but protocol is not 41
0x35	V6TRANSIT SHORT PAYLOAD	2	mode 0x2 but payload is smaller than a 40-byte IPv6 header
0x36	V6TRANSIT INNER HOPLIMIT ZERO	7	inner IPv6 Hop Limit is already zero at egress decap; cannot decrement

Code 0x36 handles the one TTL edge case the main document implies but does not state: if an IPv6 packet arrives at the fabric with Hop Limit 1, decrementing it at egress produces zero. The packet is dropped at the far edge, which is the correct IPv6 behavior (the packet would have expired on the next real hop anyway). Because transit is one hop, this can only happen when the inner packet was already at its last permitted hop on entry.

The existing RESERVED_NONZERO code (0x03) still covers the 16 reserved bits at bytes 14-15. BAD_TRANSIT_MODE (0x32) is separate because the reserved values of a used field are a different diagnostic signal from a stray bit in a reserved field.

Appendix G — Revised Header-Size Claims Across the Series

Every size claim in HOWL-NET-1 through 3 that the 36-byte correction touches, with the old value, the corrected value, and whether the surrounding argument survives.

Claim location	Old statement	Corrected	Argument survives?
NET-1 §4.1	IP header fixed at 32 bytes	36 bytes	yes; “fixed, no options” is the real point
NET-1 §4.5	12 bytes smaller than IPv6	4 bytes smaller	yes, weakened
NET-1 §9	8 bytes smaller than IPv6	4 bytes smaller	yes, weakened

Claim location	Old statement	Corrected	Argument survives?
NET-2 §4.5.3	IP+TCP 56 bytes, 8 payload bytes in cache line	60 bytes, 4 bytes margin	partially; still one cache line, no payload headroom equal to IPv6, not smaller
NET-2 §4.8.1	IPv4-64 IP+TCP 56 bytes	60 bytes	yes; still 2 bytes under IPv6
NET-2 §4.8.2	IP+UDP 42 bytes	46 bytes	yes, reduced to one third
NET-2 App P	6 bytes saved per UDP packet vs IPv6	2 bytes saved per UDP packet	yes, weakened
NET-3 §8	single-cache-line headers, 8 spare	one cache line, 0–4 spare	struct size only
NET-3 App A	ipv4 64 h total 32 bytes	36 bytes	

The bandwidth-at-scale tables in NET-2 Appendix P should be recomputed at 2 bytes saved per UDP packet instead of 6, and 0 bytes saved per TCP packet instead of 4. The direction of every argument holds; the magnitudes shrink. IPv4-64 is still smaller than IPv6 on UDP and equal on TCP, and the performance case never rested on header size — it rested on fixed offsets and no per-hop checksum, both unchanged.

Appendix H — Corrected Bandwidth Savings (replaces NET-2 Appendix P magnitudes)

Recomputed at the 36-byte header. UDP saving versus IPv6 is 2 bytes per packet (46 vs 48). TCP saving versus IPv6 is 0 bytes per packet (60 vs 60).

Traffic profile	Packets/sec	Per-packet saving vs IPv6	Saving/sec	Saving/day
100,000 VoIP calls (UDP)	5,000,000	2 bytes	10 MB/s	864 GB
1,000,000 game sessions (UDP)	60,000,000	2 bytes	120 MB/s	10.4 TB
10 billion IoT devices (UDP)	10,000,000,000	2 bytes	20 GB/s	1.73 PB
HTTP/2 small frames (TCP)	1,000,000/server	0 bytes	0	0
Bulk transfer (TCP)	81,274/Gbps	0 bytes	0	0

The UDP small-packet case remains the only place header size matters, and it still favors IPv4-64. On TCP the two protocols are now equal in size, so the IPv4-64 case on TCP rests entirely on the fixed header, the absent per-hop checksum, and the security fields, none of which depend on byte count.

Appendix I — Why Not 34 Bytes

The field widths sum to 268 bits, which is 33.5 bytes, so 34 bytes is the smallest whole-byte header that holds them. 36 was chosen over 34 for three reasons.

A 34-byte header ends the two 64-bit addresses on an odd 2-byte boundary relative to the 32-bit words that precede them, forcing the address fields to straddle 32-bit word lines. ASIC and P4 pipelines read in 32-bit words; a field that straddles a word line needs an extra read or a shifter. 36 bytes is nine clean 32-bit words and every address field lands whole inside words.

A 34-byte header leaves only 6 spare bits ($272 - 268$ would be 4, and 34 bytes is 272 bits, so 4 spare bits), not enough for the 4-bit `transit_mode` plus any reserved room. 36 bytes gives 20 spare bits: 4 for `transit_mode` and 16 held in reserve for future use without another header revision.

A 34-byte header saves 2 bytes over 36 but gives up word alignment and all future headroom. The 2 bytes do not change the overhead class (2.27% vs 2.40% on a 1500-byte MTU) and do not change cache-line behavior. The alignment and the headroom are worth more than the 2 bytes.

Appendix J — Field Origin and Status After Correction

Connects each field to where it came from in the series and whether this document changed it.

Field	Introduced in	Changed here	How
version	NET-1	no	
dscp, ecn	NET-1 (from IPv4)	no	
flags	NET-1	no	
payload length	NET-1 (from IPv6 model)	no	now counts bytes after 36, not 32
flow label	NET-1 (from IPv6)	no	also now the ECMP input for v6 transit
ttl	NET-1 (from IPv4)	no	gains fabric-only role in transit mode
protocol	NET-1 (from IPv4)	no	value 41 now signals v6 transit

Field	Introduced in	Changed here	How
identification	NET-1	no	
fragment token	NET-1	no	
fragment offset	NET-1	no	
transit mode	NET-4 (this doc)	new	4 bits at byte 13
reserved	NET-4 (this doc)	new	16 bits, must be zero
svt	NET-1	no	in transit, attests the ingress edge
src addr, dst addr	NET-1	no	dst addr now also holds a v6 /64 prefix in transit

Three existing fields take on a second role in transit mode without any change to their format: protocol (value 41 marks transit), ttl (becomes the fabric-only loop guard), and dst_addr (holds the IPv6 /64 prefix). This is the economy of the design — the new capability needed one new 4-bit field and three reinterpretations, not a new header.

Appendix K — Minimal Packet Sizes Per Mode

The smallest valid packet in each mode, for the runt-check in validation Phase 2.

Mode	Smallest valid payload	Smallest total packet	Runt rule
0x0 NATIVE, TCP	24-byte TCP header	$36 + 24 = 60$ bytes	drop if < 60
0x0 NATIVE, UDP	10-byte UDP header	$36 + 10 = 46$ bytes	drop if < 46
0x1 V4 COMPAT, TCP	24-byte TCP header	60 bytes	same as native
0x1 V4 COMPAT, UDP	10-byte UDP header	46 bytes	same as native
0x2 V6 TRANSIT	40-byte IPv6 header, no payload	$36 + 40 = 76$ bytes	drop if < 76 (code 0x35)

The transit minimum is larger because the smallest thing a transit packet can carry is a complete IPv6 header. A mode 0x2 packet shorter than 76 bytes cannot contain a valid inner IPv6 header and is dropped at validation, before any forwarding work.

These appendices assume the main HOWL-NET-4 document and the three prior docu-

ments. They add the per-byte map, the two conversion field maps, the unified routing-table behavior, the new drop codes, the corrected size claims and bandwidth figures, the 34-versus-36 reasoning, and the per-mode minimum sizes. Nothing here restates the mode definitions or the transit one-hop rule from the main document; it carries the detail those rules depend on.

[[HOWL-NET-4-2026](#)] IPv4-64 Corrected Header and Unified Routing Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-4-2026>

[[HOWL-NET-1-2026](#)] IPv4-64: A 64-Bit In-Place Upgrade to IPv4 with Modern Security and Performance. Github: <https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-1-2026>

[[HOWL-NET-2-2026](#)] IPv4-64: Security and Performance Analysis. Github: <https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-2-2026>

[[HOWL-NET-3-2026](#)] IPv4-64 Reference Implementation on P4 Programmable Hardware. Github: <https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-3-2026>